

Enhancing IoT Intrusion Detection through Vertical Federated Learning and Latent Space Mapping

1st Krishna Yadav
SAI Lab
India
krishna.nitkkr1@gmail.com

2nd Harshit Chaubey
Department of Computer Science
IIIT Naya Raipur
Raipur, India
harshit21100@iiitnr.edu.in

3rd Rajesh Sah
SAI Lab
USA
rajeshsah.sai@gmail.com

Abstract—Federated learning has emerged as a promising solution for collaborative learning of IoT-based attacks without requiring data sharing. However, the system heterogeneity among IoT devices results in diverse feature spaces due to the nature of the data generated. This heterogeneity arises from varying device types, protocols, and data formats, which lead to differences in how attacks manifest across devices. As a result, traditional FL systems struggle to align these differing feature spaces, impacting model convergence and performance. While some common attacks exhibit diverse feature spaces, they share fundamental characteristics. To address the issue of diverse feature spaces, we utilized an autoencoder at the client side to find an optimal latent node that captures the similarities between various attacks, thereby transforming heterogeneous feature spaces into a uniform latent representation. We focused on four attacks common to both the CICIDS 2017 and UNSW datasets. Initially, we trained two separate models on these attacks to evaluate individual model performance in a federated setting. Subsequently, we trained a single model using the uniform latent representation and found that this model outperformed the individual models in six out of eight classification instances. We further demonstrated that creating a uniform latent space not only resolves the issue of diverse feature spaces but also enhances the model's ability to learn shared attack characteristics, improving the classification accuracy of the federated global model.

Index Terms—Autoencoder, Federated learning, Uniform latent representation, Diverse feature space

I. INTRODUCTION

The rapid growth of the IoT in recent years has made it more challenging to design network architectures. Nowadays, given the existence of billions of devices, data is shown to be generated over layers of the OSI (Open Systems Interconnection), with each layer providing different peculiarities [1]. The OSI model consists of seven layers, ranging from the physical layer to the application layer, which together define the process of data distribution. These layers are highly diverse. For example, at the physical layer, features like signal strength and transmission rate are crucial, while at the application layer, factors such as user behavior or encryption patterns may be embedded in the data. Managing an intrusion detection system becomes challenging when threat features vary across layers within the same structure, as each layer has different characteristics and data patterns [2]. Therefore, building an effective intrusion detection system (IDS) requires advanced

problem-solving skills to handle diverse data streams and detect anomalies or malicious activity across different layers.

Federated Learning (FL) has been proven as an effective means for enhancing the performance of intrusion detection in networks of IoT [3]. This is due to the fact that it maintains the privacy of the users. Instead of relying on traditional centralized approaches that require a central repository of information, IoT leverages federated learning (FL), where machine learning models are trained locally on devices, and only model updates are sent to a central server. This preserves sensitive information, enhancing privacy by preventing the transmission of raw data. Additionally, FL allows for continuous model updates directly on IoT devices, ensuring that models can adapt to the constantly changing IoT environments without interruption. The distributed architecture also reduces bandwidth consumption and latency, key challenges in networks with numerous connected IoT devices. However, even with FL, developing effective IDS systems requires models capable of handling diverse attack patterns and variations, while still performing efficiently on low-specification devices [4].

The major dimensionality of data heterogeneity particularly associated with FL-based intrusion detection in IoT networks is the central feature endemic in heterogeneous IoT networks. This is commonly referred to as vertical federated learning in Federated Learning [5]. Heterogeneity in IoT devices ranges from high-end machines to low power consuming sensors and this variety affects the data generated. For example, edge computing devices analyze network packets while sensors capture environmental data resulting in rich data features. That is, the creation of a global model and its training becomes a difficult battle as most of the local models tend to be dissimilar making it hard to combine them efficiently. Additionally quality of the data possessed by the individual devices will vary greatly leading to differences in how well the model performance will be in practice making it hard for FL based ids to yield very good results [6]. To deal with this problem, it is necessary to create models, which while observing the privacy and efficiency of FL, embed these deviations.

Although the IoT systems are diverse in shape and structure, common attacks, and cybercrime that include denial-of-service (DoS), man-in-the-middle as well as malware injection, apply

to different devices. Such uniformity in attack patterns [7] aids in the improvement of uniform latent representation within intrusion detection in federated IoT networks. The uniform latent representation framework permits the model to learn associated tasks such as attack detection and OSI layer monitoring concurrently without independently learning not to share substance across the tasks. Being able to use similarity of the attacks, uniform latent representation improves generalization of the model across different IoT attacks, increasing the detection capability of the model even though the data on different devices is not the same. In this way, knowledge is shared when cross-device learning is utilized, and the training time or data required by the system is minimized due to improved generalization across various IoT devices and network-based attacks.

Data heterogeneity poses a significant challenge in intrusion detection for IoT networks, as various devices generate data with distinct feature spaces. This diversity complicates the design of a unified detection model that can cover the entire network. Autoencoders address this issue by using latent variables to map diverse feature spaces into a common one [8]. They achieve this by compressing input signals into a latent space, then reconstructing them to identify prominent features.

Incorporating autoencoders within the Federated Learning (FL) framework is advantageous for handling diverse data across IoT devices. Each device can encode its unique data, allowing for the use of a consistent feature set in intrusion detection. This transformation is crucial because it enables uniform feature representation, even when data is generated differently across devices. Uniformity is essential for the global model in FL, which relies on consistent feature extraction and anomaly detection across devices with varying contexts.

Autoencoders also contribute to the creation of a uniform latent representation, which improves performance by reducing data dimensionality—a key advantage for resource-constrained IoT devices. This reduction simplifies the learning process, enhances multitasking capabilities, and helps the global model focus on relevant features, leading to more accurate and reliable intrusion detection. By combining autoencoders with federated learning, an efficient and flexible intrusion detection system can be developed to tackle the diverse challenges of IoT environments.

The primary contributions of this paper are:

- We employed autoencoders to map heterogeneous feature spaces into a uniform latent representation, facilitating the training of a federated learning-based Intrusion Detection System (IDS) model.
- We introduced a voting-based mechanism to determine the optimal number of latent nodes, which identifies the best latent representation for diverse attack types.
- We analyzed attack behaviour within the uniform latent space and demonstrated that grouping interrelated attack types across different nodes enhances detection accuracy.

II. RELATED WORK

FL enhances privacy in IoT-based intrusion detection by allowing local model training without sharing raw data. This approach protects sensitive information and enables regular model updates without centralizing data. The authors [9] present a thorough overview of the situation regarding intrusion detection systems. FL is structured into Horizontal Federated Learning (HFL), Vertical Federated Learning (VFL), and Federated Transfer Learning Center based on the data distribution. HFL deals with the concept where different observers share the feature spaces whereas VFL deals with the concept where different feature spaces share observations.

The issue of data heterogeneity in machine learning, on the other hand, is slowly being addressed through VFL. VFL can be conceptualized as a form of collaborative learning which lets multiple stakeholders, owning different features of the same data, collaboratively train a machine learning model while keeping the actual data intact. While classical federation is associated with dividing samples among many devices, VFL takes a slightly different approach of dividing features among several institutions, which is very essential in cases when certain institutions have particular features for the same individuals. The framework discussed in paper [10] aims at both the functionality and security requirements in VFL by the usage of federated source layers to fuse several kinds of information including dense, sparse, numerical, and categorical. In their approach, they describe a secure execution Fefe, how the federated execution is paranoid against machine wear-out attacks making the proposed system effective for privacy-focused VFL. Incorporating VFL in cloud computing, another problem is the network anomaly detection complexity along with the lack of supervised labeled data, as reported in [11]. In this case, they discuss a federated multi-task deep neural network (MT-DNN-FL) that works with privacy and addresses various tasks, including anomaly detection and traffic classification, at the same time.

Furthermore, the work in [12] confirms that vertical decompositions of neural networks are superior in image classification tasks. The model alleviates the problem of data variation and enhances classification by vertically splitting the data and employing deep learning with already trained networks. In VFL, a key problem exists in that unaligned data is under-exploited, whereby only aligned data is used in the training process. The researches in [13] have also solved this problem as they offered a clean method that was able to integrate both aligned and unaligned data thus optimizing the model performance and cut back the training time. Their data mixture approach and use of auxiliary data were able to improve the accuracy of the model by as much as 15 percent. However, to tackle the problem of unbalanced features among different parties, the authors in [14] designed an adaptive VFL framework that enhanced the training speed of the models when features are imbalanced. This method minimizes the number of local updates each party performs to restrict the international convergence rate and convergence computation

load.

The issue of data heterogeneity is rather common when it comes to Vertical Federated Learning (VFL). There are several approaches which have been put forth in tackling the problems which emanate from non-iid data from multiple clients, which can hamper the performance and convergence of the model. In [15], FCCL (Federated Cross-Correlation and Continual Learning) is presented in order to deal with the heterogeneity issue by constructing cross-correlation matrices from unlabeled public data, thus enhancing the model under domain adaptation. This method also alleviates the issue of distinct memory impoverishment by means of knowledge distillation when conducting local updates such that both inter-domain and intra-domain information in relation to the model is maintained but, privacy is not invaded. Also, Heterogeneous Federated Learning survey [16], reviews upload bandwidth adaptation and presents a review of research challenges such as statistical, model, communication and device heterogeneity. The taxonomy proposed in this survey provides a comprehensive review of literature on the existing techniques which are based on data, model and server in solutions to these concerns.

These approaches are crucial in solving the performance degradation caused by non-i.i.d. data across clients in real-world federated learning scenarios. HeteroSwitch, proposed in [17], addresses a unique aspect of heterogeneity that stems from system-level factors such as varying hardware and software configurations on different devices. The system-induced data heterogeneity often leads to a decrease in model accuracy and fairness. HeteroSwitch mitigates this by applying ISP transformations and SWAD techniques, dynamically adapting to the biases introduced by these hardware and software variations, resulting in more consistent performance across different devices. Another innovative solution is Virtual Homogeneity Learning (VHL) [18], which attempts to directly rectify data heterogeneity by introducing a virtual homogeneous dataset.

This synthetic dataset serves to achieve the proper balances in the diversity of feature sets among the clients without infringing on the privacy of any of them. Hypothetically, VHL increases the generalization ability of the federated learning setting and improves the speed of convergence as well, giving a new way on how to address the matter of heterogeneity in a distributed arena. Lastly, [19] presents a distribution fusion approach that is data-independent to address the non-i.i.d problem in federated learning. In FedFusion, the inadequacy of statistical data regarding a distribution component is mitigated by a variational autoencoder to optimize the parameters of the distribution fusion components for better aggregations of models with heterogeneous datasets. This strategy does not only improve the models of the localized clients but the efficiency of the global model given that sometimes the distributions of the client provided data are very dissimilar.

III. BACKGROUND AND PROBLEM FORMULATION

A. Federated Learning (FL)

Federated Learning (FL) is a decentralized approach that enables devices to collaboratively train models without sharing

raw data, enhancing privacy and security.

In our context, we trained a FL global model M across multiple IoT devices $D_1, D_2, \dots, D_n$ without sharing their raw data. Each device D_i locally trains the model on its data X_i , resulting in model parameters θ_i , which are aggregated to update the global model $\theta = \frac{1}{n} \sum_{i=1}^n \theta_i$.

B. Diverse Feature Space

Data generated from different OSI layers in IoT networks lead to feature space heterogeneity. Let's consider the data on each device X_i has different feature sets F_i , making $F_1 \neq F_2 \neq \dots \neq F_n$. This variation complicates the aggregation of local models, leading to suboptimal global performance. The challenge lies in aligning these diverse feature spaces to ensure the global model performs consistently across all devices.

Let us formalize the problem of heterogeneous feature spaces in the context of federated learning for IoT:

- 1) Heterogeneity in Feature Sets: Due to the differences in $\mathcal{F}_i$, the model f_θ cannot directly operate on input X_i from client i because $X_i \in \mathcal{F}_i$ varies across clients. This necessitates a mapping mechanism to a common feature space:

$$X_i \xrightarrow{A_i} Z_i \quad \text{where} \quad Z_i \in \mathcal{Z}$$

- Here, A_i is the autoencoder specific to client i , mapping X_i to a common latent space $\mathcal{Z}$.

- 2) Feature Space Mismatch: Each client i has a unique feature space $\mathcal{F}_i$, which leads to a corresponding data distribution $\mathcal{P}_i(X, Y)$ over the input-output pairs (X, Y) . The task is to find a global model θ that minimizes the expected loss across all clients:

$$\min_{\theta} \sum_{i=1}^N w_i \mathbb{E}_{(X, Y) \sim \mathcal{P}_i} [\ell(f_\theta(X), Y)]$$

where:

- f_θ is the global model with parameters θ ,
- $\ell(\cdot)$ is the loss function (e.g., cross-entropy for classification),
- w_i is the weight for client i .

Given the diverse feature spaces across IoT devices, the goal is to develop a robust global model that effectively handles the heterogeneity in feature sets. By mapping each device's local data to a common latent space, we aim to address the feature space mismatch and ensure consistent performance. The primary objective is to train a global model whose accuracy surpasses the performance of any local model trained individually on client-specific data, thereby improving the overall effectiveness of federated learning in the presence of heterogeneous data distributions.

IV. PROPOSED APPROACH

This section will explain how we address the problem of multiple feature spaces in IoT devices using autoencoders. The approach involves three main components. First, we will

discuss the theory behind autoencoders and how they help unify different feature spaces into a common latent space. Next, we will describe how a distance-based metric can group similar attack types once the features are unified. Grouping these similar attacks can improve overall detection accuracy by allowing the model to better generalize across diverse IoT environments.

A. Autoencoder

An autoencoder is a type of neural network used for unsupervised learning to reduce dimensionality while preserving essential data features. It consists of two parts: the encoder and the decoder. The encoder compresses the input data X_i from its original feature space $\mathcal{F}_i$ into a smaller latent space Z_i , while the decoder reconstructs the data back from the latent space. The autoencoder learns to minimize the reconstruction error, ensuring that the latent representation captures the most relevant features of the data.

Mathematically, let $X_i \in \mathcal{F}_i$ represent the input data from device i , and $Z_i \in \mathcal{Z}$ represent the compressed latent representation. The encoder function f_{enc} maps the input data to the latent space:

$$Z_i = f_{\text{enc}}(X_i) \quad \text{where} \quad Z_i \in \mathcal{Z}.$$

The decoder function f_{dec} reconstructs the data back into the original feature space:

$$\hat{X}_i = f_{\text{dec}}(Z_i) \quad \text{where} \quad \hat{X}_i \approx X_i.$$

The training objective of the autoencoder is to minimize the reconstruction loss:

$$\mathcal{L} = \min_{i=1}^N \sum_{i=1}^N \|X_i - \hat{X}_i\|^2 = \min_{i=1}^N \sum_{i=1}^N \|X_i - f_{\text{dec}}(f_{\text{enc}}(X_i))\|^2.$$

By compressing data from different IoT devices into a common latent space $\mathcal{Z}$, autoencoders effectively unify the feature spaces from various devices, ensuring that the data can be processed consistently in the federated learning framework.

- 1) *Client-Specific Encoding*: Each client i trains its own autoencoder, which learns to map the device-specific feature space $\mathcal{F}_i$ to the shared latent space $\mathcal{Z}$. For each client i , the encoder function f_{enc}^i maps input features $X_i \in \mathcal{F}_i$ to the latent space:

$$Z_i = f_{\text{enc}}^i(X_i) \quad \text{where} \quad Z_i \in \mathcal{Z}.$$

Although the input feature spaces $\mathcal{F}_i$ are different across clients, the autoencoders are designed such that their outputs Z_i lie within the same latent space $\mathcal{Z}$. Mathematically, we aim to map these diverse feature spaces into a common latent space where:

$$Z_1 = Z_2 = \dots = Z_n = Z \in \mathcal{Z}.$$

This means that, for all clients, the encoded representations are aligned within the same latent space $\mathcal{Z}$, ensuring consistency for aggregation in the global federated model:

$$f_{\text{enc}}^1(X_1) = f_{\text{enc}}^2(X_2) = \dots = f_{\text{enc}}^n(X_n) = Z.$$

Here, Z represents the unified latent representation shared across all clients despite their differing input feature spaces $\mathcal{F}_i$.

- 2) *Training the Global Model*: With the encoded representations Z_i from each client, the global federated model is trained on the consistent feature space $\mathcal{Z}$. This effectively addresses the problem of heterogeneous feature spaces, allowing the global model to learn from a common representation that encapsulates the diverse inputs from all clients:

$$\min_{\theta} \sum_{i=1}^N w_i \mathbb{E}_{(Z_i, Y) \sim P_i} [\ell(f_{\theta}(Z), Y)].$$

- 3) *Latent Space Consistency*: The latent space $\mathcal{Z}$ serves as a shared representation where data from various devices, with differing original feature spaces $\mathcal{F}_i$, can be uniformly projected. This guarantees that the devices utilizing the global federated learning would always be fed the same data, no matter how heterogeneous the raw data across all the devices may be. The success of the operation of the global model depends primarily on this latent space representation where local peculiarities have been stripped away so that updates from a number of clients can be easily assimilated. However, this latent space is dependent on how the architecture of the autoencoder is designed, most importantly the quantity of nodes involved in the latent layer. In this way, if the quantity of nodes is so low, a lot of useful information from every device in relation to the dataset would be missed out, and this will result to poor performance of the global model. On the other hand, if the latent space size is too large, it might be able to generalize but still worse, it would only be successful using data from specific devices. For the latent space to optimally represent relevant information on various datasets, adjusting the sizes of the nodes in the latent space is important. This optimization enables the latent space to provide a proper trade-off between retaining essential features of different datasets and generalizing for the global federated learning model to be effective across all devices.
- 4) *Optimizing the Latent Representation*: One key aspect of employing the autoencoders is the determination of the optimal number of latent representation nodes, l . If l is too small, there may be a loss of important information and if it is too large, overfitting or high computational overhead may occur. In order to determine the appropriate number of visible nodes, it is necessary to make a use of GridSearchCV which is a brute-force technique rastering from a lower number of l and moving towards

higher numbers rather than from 10 to 80 nodes. This method consists in changing all the values of l counting on the one that gives the lowest value of the reconstruction error and generalizes properly.

B. Intrusion under latent representation

1) *Grouping Similar Attacks Using Unified Features and Distance-Based Metrics*: Once the autoencoder maps different feature spaces from IoT devices into a common latent space, it becomes possible to compare and group similar attack types across devices. In this unified feature space, attacks with similar characteristics can be grouped together, allowing for more effective detection and classification by focusing on patterns shared across the network. By leveraging a distance-based metric, we can quantify the similarity between the latent representations of attacks, facilitating the grouping process.

2) *Grouping Similar Attacks*: After the autoencoder transforms the input features X_i from each device into the latent space Z_i , we can use a distance metric, such as Euclidean distance or cosine similarity, to measure the closeness between different attacks. Mathematically, for two latent representations Z_i and Z_j corresponding to two different attacks, the Euclidean distance can be defined as:

$$d(Z_i, Z_j) = \|Z_i - Z_j\|_2 = \sqrt{\sum_{k=1}^n (Z_i^k - Z_j^k)^2}$$

where Z_i^k and Z_j^k are the k -th dimensions of the latent representations for attack i and attack j , respectively. A smaller distance implies a higher similarity between the attacks, while a larger distance indicates dissimilarity.

3) *Clustering Attacks Based on Similarity*: Once the distances between all attack pairs are calculated, clustering algorithms such as k-means can be used to group similar attacks into clusters. For example, using k-means clustering, we partition the latent space $\mathcal{Z}$ into k clusters, where attacks in the same cluster are closer to each other in terms of the distance metric. The objective function for k-means clustering can be written as:

$$\min_{C_1, C_2, \dots, C_k} \sum_{i=1}^k \sum_{Z_j \in C_i} \|Z_j - \mu_i\|^2$$

where C_i is the i -th cluster and μ_i is the centroid of cluster C_i . This minimizes the intra-cluster distance, ensuring that similar attacks are grouped together.

4) *Federated Averaging with Autoencoder*: The modified vertical federated learning algorithm presented in Algorithm 1 incorporates an autoencoder to align the feature spaces of diverse clients. In the first round, each client i trains an autoencoder AE_i on their local data X_i and performs a grid search over the list of potential latent node sizes $L = [l_1, l_2, \dots, l_k]$ to find the best number of latent nodes l_i^* . The clients then send these optimal latent node sizes l_i^* to the server, which determines the best size l^* through a voting mechanism $l^* = \text{Mode}(\{l_i^*\}_{i=1}^N)$. In subsequent rounds, clients

Algorithm 1 Modified Vertical Federated Learning with Autoencoder

Require:

Number of clients: N

Number of rounds: R

List of potential latent node sizes: $L = [l_1, l_2, \dots, l_k]$

Ensure:

Final global model M

0: Initialization:

0: **for** each client i in $\{1, 2, \dots, N\}$ **do**

0: Initialize autoencoder A_i

0: **end for**

0: Round 1: Autoencoder Training and Latent Node Selection

0: Server sends the autoencoder model A_i to all clients.

0: **for** each client i in $\{1, 2, \dots, N\}$ **do**

0: $A_i \leftarrow \text{TrainAutoencoder}(X_i, A_i)$

0: $l_i^* = \arg \min_{l \in L} \text{ReconstructionError}(A_i, X_i, l)$

0: Send l_i^* to the server.

0: **end for**

0: $l^* = \text{Mode}(\{l_i^*\}_{i=1}^N)$

0: Subsequent Rounds: Federated Learning with Uniform Features

0: **for** each client i in $\{1, 2, \dots, N\}$ **do**

0: $Z_i = \text{TransformData}(X_i, A_i, l^*)$

0: **end for**

0: **for** each round r from 2 to R **do**

0: **for** each client i in $\{1, 2, \dots, N\}$ **do**

0: $M_i \leftarrow \text{TrainLocalModel}(Z_i, M_i)$

0: $\Delta M_i = M_i - M$

0: Send ΔM_i to the server.

0: **end for**

0: $M \leftarrow M + \frac{1}{N} \sum_{i=1}^N \Delta M_i$

0: Server sends the updated global model M back to all clients.

0: **end for=0**

use the selected latent node size l^* to transform their data X_i into a uniform feature space Z_i . Federated learning then proceeds with clients training their local models M_i on the transformed data Z_i and sending model updates ΔM_i to the server. The server aggregates these updates to form a global model M , which is then sent back to the clients. This process continues for a specified number of rounds R , resulting in a final global model M .

5) *Impact on Intrusion Detection Accuracy*: Grouping similar attacks based on their latent representations offers two key advantages for improving the accuracy of intrusion detection systems. First, it enhances generalization. By learning shared representations of similar attacks, the system can better generalize to new, unseen attacks. Even if new attacks do not exactly match previous patterns, they can still be detected by belonging to a cluster of known attacks, allowing analysts to recognize and address potential threats more effectively.

Second, this approach improves the efficiency of model

training. By categorizing attacks of a similar nature, the system minimizes the need to develop separate models for every individual attack type. Instead, the model can focus on learning generalized characteristics for each cluster of attacks, which simplifies training and reduces the computational overhead.

V. EXPERIMENTATION

This section will describe the experimentation setup, including datasets, results, and various analyses. We begin with the dataset description and present the results of our models under both individual datasets and federated learning settings. First, we evaluate the performance of the models on individual datasets, followed by testing in federated settings using a uniform feature space.

A. Dataset And Description

We utilized two well-known intrusion detection datasets for our experiments: CICIDS 2017 and UNSW-NB15.

1) *CICIDS 2017 Dataset*: This dataset simulates modern network traffic with a mix of normal and malicious activities. It contains a total of 2,231,806 data points, each characterized by 78 features, including network traffic details and a class label for identifying whether the traffic is benign or malicious. The dataset covers 15 different attack types, making it a comprehensive resource for intrusion detection evaluation. The distribution of class labels includes 1,895,314 benign data points and 336,492 attack points.

2) *UNSW-NB15 Dataset*: The purpose of the UNSW-NB15 dataset is to capture more realistic network attacks focusing on a wider spectrum of threats relative to its predecessor. Also, it provides 49 features per data point and gives a label that classifies the traffic into normal or attack-type. The dataset comprises nine types of attacking strings such as Fuzzers, BackDoors, DoS, Exploits etc.

We selected two datasets with different feature spaces, assuming they originate from a heterogeneous IoT system. We identified several attack types, including DDoS, Brute Force, Reconnaissance, and Exploits, that were common across both datasets. We split the data into a training set and a validation set in an 8:2 ratio. The classification results in Table I reflect the model's performance on the validation set. For further analysis, we focused on these attacks and trained a federated learning model distributed across twenty federated nodes. To assess the similarity between the attack types, we employed clustering-based metrics, specifically analyzing inter- and intra-cluster distances. The clustering analysis revealed that, although these attacks originate from different devices, they share fundamental characteristics. These shared characteristics can be leveraged to create more robust latent representations, improving the accuracy of classifying similar attack types.

B. Federated Training Results on Individual dataset

Initially, we trained the FL model on the individual CICIDS and UNSW datasets to evaluate its performance across different attack types. The results for the CICIDS dataset were promising, particularly for benign traffic, which achieved

nearly perfect scores with a precision of 0.996, recall of 0.998, and an F1-score of 0.997. For DDoS, the model also performed well, with an F1-score of 0.980, indicating strong detection capability. However, certain underrepresented attack types, such as Exploits, were difficult for the model to detect, resulting in an F1-score of just 0.172. This indicates that while the model excels at detecting common attack types, it struggles with rarer ones.

In contrast, the model's performance on the UNSW dataset was more varied. For benign, the model performed reasonably well with an F1-score of 0.954. However, the model struggled with DDoS and Bruteforce, achieving F1-scores of 0.705 and 0.698, respectively. These lower scores suggest that the model had difficulty generalizing across certain attack types in the UNSW dataset. Interestingly, the model's performance in detecting Exploits improved significantly, with a recall of 0.837 and an F1-score of 0.736, indicating better detection of complex or rare attacks in this dataset than CICIDS.

C. Results on Grouping Similar Attacks

To address the limitations of individual dataset training, we applied a distance-based clustering approach to group similar attacks across both CICIDS and UNSW datasets. By clustering similar attack types based on their proximity in the latent space, we aimed to improve the model's generalization ability across different types of attacks. We first applied dimensionality reduction techniques such as Principal Component Analysis (PCA) followed by k-means clustering to visualize and group similar attacks effectively.

We measured two types of distances: intra-cluster and inter-cluster. Intra-cluster distance refers to the distance between the same types of attacks across two datasets. For example, it measures the distance between the DDoS attack in the CICIDS dataset and the DDoS attack in the UNSW dataset. On the other hand, inter-cluster distance is defined as the distance between a specific attack type, like DDoS, and all other attack types. For intra-cluster distance, we computed the average across multiple measurements. We used the elbow method to determine the optimal number of clusters, which is 4. The method involves plotting the variance explained as a function of the number of clusters, and selecting the point where adding more clusters no longer significantly reduces variance. As demonstrated in Figure 1, intra-cluster distances were significantly smaller than inter-cluster distances, particularly in the UNSW dataset, confirming that similar attacks were grouped closely in the latent space.

DDoS and Bruteforce, for example, showed a clear distinction between the smaller intra-cluster distances and larger inter-cluster distances, supporting the effectiveness of the clustering approach. This separation allowed the model to detect attack patterns more accurately, improving classification performance. By grouping similar attacks, the model could generalize better across both datasets, enhancing its ability to detect common and rare threats in IoT networks.

TABLE I
METRIC SCORES FOR CICIDS, UNSW, AND UNIFORM LATENT SPACE ACROSS DIFFERENT CLASSES

Class	CICIDS [Precision, Recall, F1-Score]	UNSW [Precision, Recall, F1-Score]	Uniform Latent Space(CICIDS) [Precision, Recall, F1-Score]	Uniform Latent Space(UNSW) [Precision, Recall, F1-Score]
Benign	[0.996, 0.998, 0.997]	[0.944, 0.965, 0.954]	[0.997, 0.998, 0.998]	[0.981, 0.988, 0.984]
DDoS	[0.984, 0.978, 0.981]	[0.925, 0.814, 0.866]	[0.986, 0.988, 0.984]	[0.934, 0.851, 0.890]
Bruteforce	[0.985, 0.958, 0.971]	[0.743, 0.671, 0.705]	[0.977, 0.928, 0.951]	[0.800, 0.707, 0.750]
Reconnaissance	[0.958, 0.860, 0.906]	[0.787, 0.628, 0.699]	[0.968, 0.890, 0.927]	[0.827, 0.583, 0.684]
Exploits	[0.969, 0.095, 0.172]	[0.657, 0.837, 0.736]	[0.969, 0.212, 0.337]	[0.651, 0.851, 0.738]

TABLE II
LOSSES OF CLIENTS FOR EACH NODE

Client	10 Nodes	20 Nodes	30 Nodes	40 Nodes	50 Nodes	60 Nodes	70 Nodes	80 Nodes
C1	0.0575	0.0428	0.0524	0.0317	0.0168	0.0465	0.0429	0.0782
C2	0.0144	0.0125	0.0109	0.0108	0.0039	0.0097	0.0109	0.0099
C3	0.0437	0.0532	0.0448	0.0513	0.0332	0.0660	0.0373	0.0453
C4	0.1981	0.1986	0.1915	0.1869	0.1790	0.2066	0.1977	0.1821
C5	0.0947	0.0752	0.0768	0.0671	0.0514	0.0834	0.0667	0.0717
C6	0.0661	0.0629	0.0623	0.0555	0.0445	0.0605	0.0853	0.0604
C7	0.0241	0.0184	0.0175	0.0165	0.0157	0.0139	0.0170	0.0161
C8	0.0186	0.0143	0.0133	0.0134	0.0131	0.0131	0.0163	0.0137
C9	0.1681	0.1702	0.1883	0.1508	0.1396	0.1474	0.1691	0.1482
C10	0.0432	0.1315	0.0404	0.0339	0.0310	0.0365	0.0322	0.0434
C11	0.0455	0.0395	0.0385	0.0309	0.0299	0.0321	0.0325	0.0320
C12	0.0423	0.0375	0.0303	0.0324	0.0284	0.0304	0.0294	0.0330
C13	0.1118	0.0868	0.0807	0.0768	0.0714	0.0767	0.0830	0.0767
C14	0.1602	0.1414	0.1252	0.1113	0.1220	0.1062	0.1189	0.1116
C15	0.1072	0.0734	0.0646	0.0586	0.0516	0.0597	0.0543	0.0535
C16	0.0613	0.0515	0.0430	0.0393	0.0387	0.0422	0.0397	0.0404
C17	0.0561	0.0542	0.0475	0.0437	0.0430	0.0390	0.0489	0.0408
C18	0.0846	0.0719	0.0802	0.0793	0.0638	0.0662	0.0646	0.0738
C19	0.0879	0.0750	0.0718	0.0660	0.0581	0.0624	0.0636	0.0581
C20	0.0769	0.0614	0.0548	0.0573	0.0489	0.0524	0.0499	0.0529

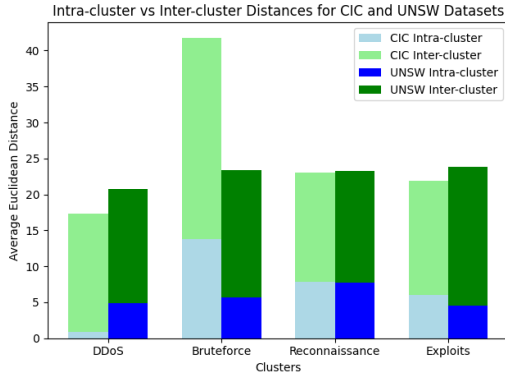


Fig. 1. Clustering results on CICIDS and UNSW Datasets

D. Results on Uniform Latent Representation

We employed autoencoders to bring the different feature spaces into a unified latent representation to further improve model performance and address data heterogeneity between the two datasets. This approach allowed the model to process the datasets in a common feature space, making learning more consistent across devices. To determine the optimal number of nodes by autoencoder as described in Algorithm 1, we perform a grid search by passing nodes ranging from 10 to 80 to the

clients, and the client sends losses over all the nodes to the central server. The losses over different nodes is presented in Table II. We divided the CICIDS 2017 data across the first ten nodes and UNSW data across the rest 10 nodes.

As shown in Table II, the configuration with 50 nodes consistently produced the lowest loss for most clients, particularly for clients such as C1 = 0.0168, C2 = 0.0039, and C10 = 0.0310. After analyzing the results and voting across multiple clients, we selected 50 nodes as the optimal configuration. This setting struck the right balance between low reconstruction loss and computational efficiency, allowing the autoencoder to build a robust unified latent representation that improved the model's generalization across both datasets.

E. Federated Learning Results with Uniform Latent representation

The results from training a federated learning model on a uniform latent representation are presented in I. The first and second columns show the precision, recall, and F1-score for the CICIDS and UNSW datasets, respectively, when individual models were trained on each dataset. The third and fourth columns represent the model trained on the uniform latent representations obtained from both datasets but validated separately on CICIDS 2017 and UNSW.

We observe that the F1-score for the DDoS class in CICIDS 2017 improved from 0.981 to 0.982 when trained on the

uniform latent representation, while in UNSW, it increased from 0.866 to 0.890. Similarly, for underrepresented attacks like Reconnaissance, the F1-score for CICIDS increased from 0.906 to 0.927, although it slightly decreased in UNSW, from 0.699 to 0.684.

Another example is the Exploit attack, an underrepresented sample, where the F1-score saw a significant increase in CICIDS, rising from 0.172 to 0.337, and in UNSW, from 0.337 to 0.738. This boost in F1-score can be attributed to the uniform latent representation improving the model's generalization, enabling it to capture shared features more effectively.

From the results in Table 1, it can be concluded that underrepresented samples, such as Reconnaissance and Exploit attacks, benefit greatly from the shared uniform latent representation, resulting in significant performance improvements. Overrepresented samples like DDoS also benefit from this approach. These findings emphasize the advantages of using a unified latent space in federated learning environments, as it enables better generalization across multiple datasets and enhances overall model performance in detecting common and rare attacks.

VI. CONCLUSION AND FUTURE WORK

In this paper, we addressed the challenge of diverse feature spaces resulting from heterogeneous IoT architectures. To solve this, we transformed the diverse feature spaces into a homogeneous space by introducing a uniform latent representation, achieved by sending an additional autoencoder model in the initial round of federated training. We demonstrated that the federated model trained on this uniform latent representation outperforms individual models in attack classification. Though the proposed uniform latent representation approach improves generalization and detection accuracy across diverse datasets, it has some limitations. The benefits of using a latent space include consistent feature mapping and better clustering of similar attacks; however, it may also lead to the loss of critical information specific to individual datasets. Additionally, the model might struggle with rare or novel attacks that do not align well in the latent space. While sending the autoencoder ensures a uniform feature space, it introduces additional communication and computational costs in the initial training round. In future work, we aim to analyze and optimize our approach further to reduce these costs and improve its overall efficiency.

REFERENCES

- [1] Marco Lombardi, Francesco Pascale, and Domenico Santaniello. Internet of things: A general overview between architectures, protocols and applications. *Information*, 12(2):87, 2021.
- [2] Asadullah Momand, Sana Ullah Jan, and Naeem Ramzan. A systematic and comprehensive survey of recent advances in intrusion detection systems using machine learning: Deep learning, datasets, and attack taxonomy. *Journal of Sensors*, 2023(1):6048087, 2023.
- [3] Md Mamunur Rashid, Shahriar Usman Khan, Fariha Eusufzai, Md Azharuddin Redwan, Saifur Rahman Sabuj, and Mahmoud Elsharief. A federated learning-based approach for improving intrusion detection in industrial internet of things networks. *Network*, 3(1):158–179, 2023.
- [4] Hao Li, Chengcheng Li, Jian Wang, Aimin Yang, Zezhong Ma, Zunqian Zhang, and Dianbo Hua. Review on security of federated learning and its application in healthcare. *Future Generation Computer Systems*, 144:271–290, 2023.
- [5] Y. Liu, Y. Kang, T. Zou, Y. Pu, Y. He, X. Ye, Y. Ouyang, Y. Zhang, and Q. Yang. Vertical federated learning: Concepts, advances, and challenges. *IEEE Transactions on Knowledge and Data Engineering*, 36(07):3615–3634, jul 2024.
- [6] Ke He, Dan Dongseong Kim, and Muhammad Rizwan Asghar. Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 25(1):538–566, 2023.
- [7] Mark Mbok Ogonji, George Okeyo, and Joseph Muliari Wafula. A survey on privacy and security of internet of things. *Computer Science Review*, 38:100312, 2020.
- [8] Zijia Lin, Guiguang Ding, Jungong Han, and Ling Shao. End-to-end feature-aware label space encoding for multilabel classification with many classes. *IEEE transactions on neural networks and learning systems*, 29(6):2472–2487, 2017.
- [9] Shaashwat Agrawal, Sagnik Sarkar, Ons Aouedi, Gokul Yenduri, Kandara Piamrat, Mamoun Alazab, Sweta Bhattacharya, Praveen Kumar Reddy Maddikunta, and Thippa Reddy Gadekallu. Federated learning for intrusion detection system: Concepts, challenges and future directions. *Computer Communications*, 195:346–361, 2022.
- [10] Fangcheng Fu, Huanran Xue, Yong Cheng, Yangyu Tao, and Bin Cui. Blindfl: Vertical federated machine learning without peeking into your data. In *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD '22, page 1316–1330, New York, NY, USA, 2022. Association for Computing Machinery.
- [11] Ying Zhao, Junjun Chen, Di Wu, Jian Teng, and Shui Yu. Multi-task network anomaly detection using federated learning. In *Proceedings of the 10th International Symposium on Information and Communication Technology*, SolCT '19, page 273–279, New York, NY, USA, 2019. Association for Computing Machinery.
- [12] Erdong Hu, Yuxin Tang, Anastasios Kyriklidis, and Chris Jermaine. Federated learning over images: Vertical decompositions and pre-trained backbones are difficult to beat. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19385–19396, October 2023.
- [13] Yihang Cheng, Lan Zhang, Junyang Wang, Xiaokai Chu, Dongbo Huang, and Lan Xu. Fedmix: Boosting with data mixture for vertical federated learning. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3379–3392, 2024.
- [14] Jie Zhang, Song Guo, Zhihao Qu, Deze Zeng, Haozhao Wang, Qifeng Liu, and Albert Y. Zomaya. Adaptive vertical federated learning on unbalanced features. *IEEE Transactions on Parallel and Distributed Systems*, 33(12):4006–4018, 2022.
- [15] Wenke Huang, Mang Ye, and Bo Du. Learn from others and be yourself in heterogeneous federated learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10143–10153, 2022.
- [16] Mang Ye, Xiuwen Fang, Bo Du, Pong C Yuen, and Dacheng Tao. Heterogeneous federated learning: State-of-the-art and research challenges. *ACM Computing Surveys*, 56(3):1–44, 2023.
- [17] Gyudong Kim, Mehdi Ghasemi, Soroush Heidari, Seungryong Kim, Young Geun Kim, Sarma Vruthula, and Carole-Jean Wu. Heteroswitch: Characterizing and taming system-induced data heterogeneity in federated learning. *Proceedings of Machine Learning and Systems*, 6:31–45, 2024.
- [18] Zhenheng Tang, Yonggang Zhang, Shaohuai Shi, Xin He, Bo Han, and Xiaowen Chu. Virtual homogeneity learning: Defending against data heterogeneity in federated learning. In *International Conference on Machine Learning*, pages 21111–21132. PMLR, 2022.
- [19] Jian-hui Duan, Wenzhong Li, Derun Zou, Ruichen Li, and Sanglu Lu. Federated learning with data-agnostic distribution fusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8074–8083, 2023.